

September 9, 2026 at 19:37

1. Introduction. Exercise 7.2.2.1–196 is an exercise in analysis rather than in search. It takes the smallest interesting family of exact cover problems—the *bounded permutation problem*, which asks for all permutations $p_1 \dots p_n$ of $\{1, \dots, n\}$ with $p_j \leq a_j$ —and asks exactly how much work Algorithm X does on it, counted in *updates*.

196. [M29] Given a bounded permutation problem defined by $a_1 \dots a_n$, consider the “dual” problem defined by $b_1 \dots b_n$, where b_k is the number of j such that $1 \leq j \leq n$ and $a_j \geq n + 1 - k$. [Equivalently, $b_n \dots b_1$ is the conjugate of the integer partition $a_n \dots a_1$, in the sense of Section 7.2.1.4.]

- a) What is the dual problem when $n = 9$ and $a_1 \dots a_9 = 246677889$?
- b) Prove that the solutions to the dual problem are essentially the inverses of the permutations that solve the original problem.
- c) If Algorithm X begins with an a_1 -way branch on item X_1 , how many updates does it perform while preparing for the subproblems at depth 1 of its search tree?
- d) How many solutions does a bounded permutation problem have, given $a_1 \dots a_n$?
- e) Give a formula for the total number of updates, assuming that the algorithm always branches on X_j at depth $j - 1$ of the search tree.
- f) Evaluate the formula of (e) when $a_j = n$ for $1 \leq j \leq n$ (that is, all permutations).
- g) Evaluate the formula of (e) when $a_j = \min(j + 1, n)$ for $1 \leq j \leq n$.
- h) Evaluate the formula of (e) when $a_j = \min(2j, n)$ for $1 \leq j \leq n$.
- i) Show, however, that the assumption in (e) is not always correct. How can the total updates be calculated correctly in general?

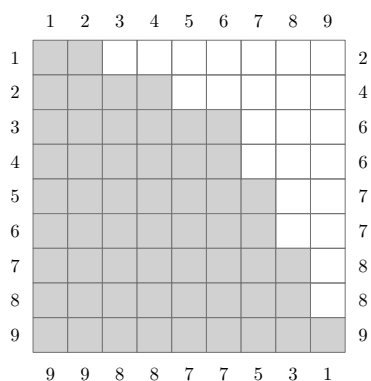
2. Nine claims, every one of them a number or a formula, and every one of them checkable. That is what this program does: it builds each problem, runs Algorithm X on it with the updates counted the way Knuth’s own DLX programs count them, and compares.

I wrote it in September 2026 while reading the exercise and its answer carefully, and it turned up three things worth reporting, all of them in the answer rather than in the exercise. Answer (h) states Π_n without its two factorial signs, though the same quantity appears with them on page 103. Answer (h) also opens its series with 6 where the term is 2. And the branching sequence quoted at the end of answer (i) is not the one Algorithm X follows.

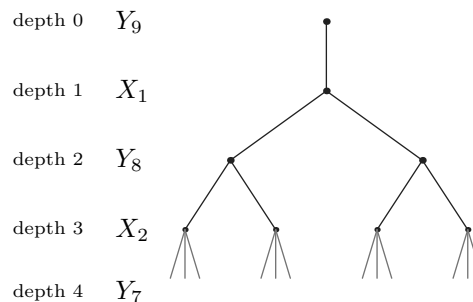
These notes, with the numbers this program produced, are the companion document `README.md` in the directory above.

3. Here is the picture the exercise draws in the reader's head. On the left, the 9×9 grid of options for $a_1 \dots a_9 = 246677889$: a cell in row j and column k means that the option ' $X_j Y_k$ ' exists, so the filled cells form the Young diagram of the partition. Count the cells in each column and read the counts from the right: they are $b_1 \dots b_9 = 135778899$, the dual problem. That is the same fact as answer (a)'s advice to draw the bipartite graph and rotate it by 180° , which exchanges the X 's with the Y 's and reverses each row. The middle panel shows which item Algorithm X actually branches on at each depth, and the right panel shows the updates per solution settling down to $4e - 1$ for two of the three families of parts (f)–(h).

the options $X_j Y_k$, and the column counts



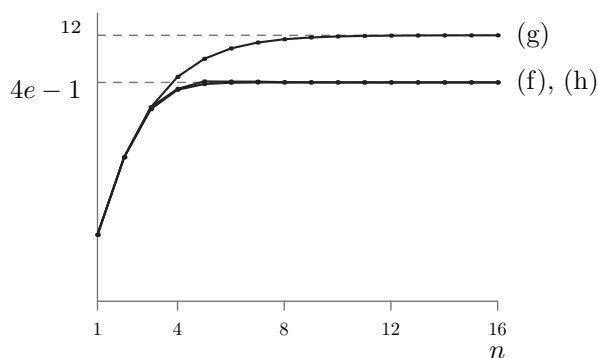
the item step X3 branches on



read from the right: $b = 135778899$

$Y_9, X_1, Y_8, X_2, \dots$ alternating

updates per solution



$4e - 1$ for (f) and (h), 12 for (g)

4. The skeleton is a command line and a choice of what to check.

```

package main
import (
    "flag"
    "fmt"
    "log"
    "math"
    "math/big"
    "sort"
    "strings"
    cells "github.com/sjnam/dancing-cells"
)
⟨ Declarations 7 ⟩
⟨ Functions 8 ⟩
func main() {
    ⟨ Read the command line 5 ⟩
    for _, m := range modes {
        switch m {
            case "dual":
                ⟨ Take the dual, part (a) 24 ⟩
            case "inverse":
                ⟨ Match the solutions with their inverses, part (b) 28 ⟩
            case "root":
                ⟨ Count the updates at the root, part (c) 29 ⟩
            case "count":
                ⟨ Count the solutions, part (d) 33 ⟩
            case "total":
                ⟨ Count all of the updates, part (e) 35 ⟩
            case "families":
                ⟨ Evaluate the three families, parts (f), (g), (h) 39 ⟩
            case "mrv":
                ⟨ Watch the heuristic choose, part (i) 43 ⟩
            case "sweep":
                ⟨ Sweep every small problem 52 ⟩
            default:
                log.Fatalf("unknown mode %q", m)
        }
    }
}

```

5. Mode `all` runs the eight checks in the order of the exercise.

⟨Read the command line 5⟩ ≡

```

mode := flag.String("mode", "all",
    "dual, inverse, root, count, total, families, mrv, sweep, or all")
digits := flag.String("a", "246677889", "the sequence a_1...a_n, one digit each")
upto := flag.Int("upto", 9, "how far the exhaustive sweeps go")
top := flag.Int("top", 16, "largest n for the three families")
flag.Parse()
modes := []string{*mode}
if *mode == "all" {
    modes = []string{"dual", "inverse", "root", "count", "total",
        "families", "mrv", "sweep"}
}
a := digitSeq(*digits)

```

This code is used in section 4.

6. Algorithm X, with its updates counted. An *update*, in the vocabulary of Section 7.2.2.1, is the removal of an option from an item's list. Knuth's programs count one for every node that *hide* unlinks and one more for every item that *cover* removes from the list of items still to be covered; DLX2 and its descendants report the total alongside the number of solutions. So the quantity this exercise analyzes is a property of the doubly linked lists themselves.

That rules out the engines in this repository. They descend from Knuth's sparse-set programs, where an item is not covered by unlinking nodes but by shortening a set, and where `ssxcc` forces items of size one before it branches at all. Their update counts are their own. To read an exercise about Algorithm X I need Algorithm X, links and all.

7. So here it is: a horizontal doubly linked list of the items still to be covered, and a vertical doubly linked list for each item, threaded through a node array with a spacer between consecutive options. The four operations that work on it are *cover*, *hide*, *uncover* and *unhide*, exactly as in (12)–(15) of the text.

⟨Declarations 7⟩ ≡

```
type dlx struct {
    items int    // how many items there are
    name []string // item names, indexed from 1
    llink, rlink []int // the active items, in a ring
    top, ulink, dlink, size []int // the node array
    opts [][]int // the options, as lists of item numbers
    optOf []int // which option each node belongs to
    path []int // the options chosen on the way down
    updates uint64 // what the exercise is about
    nodes uint64 // the size of the search tree
    count uint64 // solutions found
    prep uint64 // updates spent preparing depth 1
    leftmost bool // branch leftmost, not on the fewest
    chose []map[string]bool
    sols [][]int // the solutions, if keep
    keep bool
}
```

This code is used in section 4.

8. Item i occupies $name[i]$ and node i , for $1 \leq i \leq items$; node 0 heads the ring of active items and never moves.

⟨Functions 8⟩ ≡

```
func newDLX(names []string, opts [][]int) *dlx {
    n := len(names)
    d := &dlx{items: n, name: append([]string{" "}, names...), opts: opts}
    d.llink = make([]int, n+1)
    d.rlink = make([]int, n+1)
    for i := 0; i ≤ n; i++ {
        d.llink[i], d.rlink[i] = (i+n)%(n+1), (i+1)%(n+1)
    }
    ⟨Lay out the node array 9⟩
    return d
}
```

This code is also defined in sections 10, 12, 14, 17, 19, 20, 21, 22, 23, 25, 26, 30, 31, 32, 34, 36, 37, 38, 48, 50, 51.

This code is used in section 4.

9. Each option contributes one node per item, appended to the bottom of that item's list, and then a spacer. A spacer has a nonpositive *top*; its *ulink* points at the first node of the option above it and its *dlink* at the last node of the option below, which is what lets *hide* walk an option in a circle.

```

⟨ Lay out the node array 9 ⟩ ≡
  d.top = make([ ]int, n + 2)
  d.ulink = make([ ]int, n + 2)
  d.dlink = make([ ]int, n + 2)
  d.size = make([ ]int, n + 2)
  d.optOf = make([ ]int, n + 2)
  for i := 0; i ≤ n; i++ {
    d.ulink[i], d.dlink[i] = i, i
  }
  d.optOf[n + 1] = -1
  for k, o := range opts {
    first := len(d.top)
    for _, i := range o {
      u := d.ulink[i]
      d.top = append(d.top, i)
      d.ulink = append(d.ulink, u)
      d.dlink = append(d.dlink, i)
      d.size = append(d.size, 0)
      d.optOf = append(d.optOf, k)
      x := len(d.top) - 1
      d.dlink[u], d.ulink[i] = x, x
      d.size[i]++
    }
    d.dlink[first - 1] = len(d.top) - 1
    d.top = append(d.top, -(k + 1))
    d.ulink = append(d.ulink, first)
    d.dlink = append(d.dlink, 0)
    d.size = append(d.size, 0)
    d.optOf = append(d.optOf, -1)
  }

```

This code is used in section 8.

10. Covering item *i* hides every option that uses it and then unlinks *i* itself. The last line is the update that answer (c) counts as its leading 1.

```

⟨ Functions 8 ⟩ +=
  func (d *dlx) cover(i int) {
    for p := d.dlink[i]; p ≠ i; p = d.dlink[p] {
      ⟨ Hide the other items of the option through p 11 ⟩
    }
    l, r := d.llink[i], d.rlink[i]
    d.rlink[l], d.llink[r] = r, l
    d.updates++
  }

```

11. Hiding walks the option rightward from p , wrapping at the spacer, and takes each node out of its item's list. One update apiece.

⟨Hide the other items of the option through p 11⟩ $\equiv$

```

for  $q := p + 1; q \neq p; \{$ 
   $x := d.top[q]$ 
  if  $x \leq 0 \{$ 
     $q = d.ulink[q]$ 
    continue
   $\}$ 
   $u, w := d.ulink[q], d.dlink[q]$ 
   $d.dlink[u], d.ulink[w] = w, u$ 
   $d.size[x]--$ 
   $d.updates++$ 
   $q++$ 
 $\}$ 

```

This code is used in section 10.

12. Uncovering runs the film backwards and costs nothing, which is why the exercise can speak of the updates of a whole search as a single number.

⟨Functions 8⟩ $+ \equiv$

```

func ( $d *dlx$ ) uncover( $i$  int)  $\{$ 
   $l, r := d.llink[i], d.rlink[i]$ 
   $d.rlink[l], d.llink[r] = i, i$ 
  for  $p := d.ulink[i]; p \neq i; p = d.ulink[p] \{$ 
    ⟨Unhide the other items of the option through  $p$  13⟩
   $\}$ 
 $\}$ 

```

13. ⟨Unhide the other items of the option through p 13⟩ $\equiv$

```

for  $q := p - 1; q \neq p; \{$ 
   $x := d.top[q]$ 
  if  $x \leq 0 \{$ 
     $q = d.dlink[q]$ 
    continue
   $\}$ 
   $u, w := d.ulink[q], d.dlink[q]$ 
   $d.dlink[u], d.ulink[w] = q, q$ 
   $d.size[x]++$ 
   $q--$ 
 $\}$ 

```

This code is used in section 12.

14. Steps X2–X8. An empty item list is a solution; otherwise we choose an item, cover it, and try each of its options in turn.

⟨ Functions 8 ⟩ +≡

```

func (d *dlx) search(depth int) {
    d.nodes ++
    if d.rlink[0] ≡ 0 {
        ⟨ Record a solution 18 ⟩
        return
    }
    i := d.branchItem(depth)
    mark := d.updates
    d.cover(i)
    if depth ≡ 0 {
        d.prep += d.updates − mark
    }
    for x := d.dlink[i]; x ≠ i; x = d.dlink[x] {
        ⟨ Cover the other items of option x 15 ⟩
        d.path[depth] = d.optOf[x]
        d.search(depth + 1)
        ⟨ Uncover the other items of option x 16 ⟩
    }
    d.uncover(i)
}

```

15. Part (c) asks for exactly the updates counted here plus the *cover*(*i*) above: what it costs to have the depth-1 subproblems standing and ready, and nothing of what happens inside them.

⟨ Cover the other items of option *x* 15 ⟩ ≡

```

mark = d.updates
for p := x + 1; p ≠ x; {
    j := d.top[p]
    if j ≤ 0 {
        p = d.ulink[p]
        continue
    }
    d.cover(j)
    p ++
}
if depth ≡ 0 {
    d.prep += d.updates − mark
}

```

This code is used in section 14.

16. $\langle \text{Uncover the other items of option } x \text{ 16} \rangle \equiv$

```

for  $p := x - 1; p \neq x; \{$ 
   $j := d.top[p]$ 
  if  $j \leq 0 \{$ 
     $p = d.dlink[p]$ 
    continue
   $\}$ 
   $d.uncover(j)$ 
   $p --$ 
 $\}$ 

```

This code is used in section 14.

17. Step X3, the MRV heuristic: the item with the fewest remaining options wins, and among equals the one that comes first in the item list. That is the rule in Knuth's programs, where the scan keeps a candidate only when it is strictly smaller. Setting *leftmost* replaces the contest by the assumption of part (e), which for our item order means branching on X_j at depth $j - 1$.

$\langle \text{Functions 8} \rangle + \equiv$

```

func ( $d *dlx$ ) branchItem( $depth$  int) int {
   $best := d.rlink[0]$ 
  if  $\neg d.leftmost \{$ 
     $least := 1 \ll 30$ 
    for  $i := d.rlink[0]; i \neq 0; i = d.rlink[i] \{$ 
      if  $d.size[i] < least \{$ 
         $best, least = i, d.size[i]$ 
       $\}$ 
     $\}$ 
   $\}$ 
  if  $d.chose \neq \Lambda \wedge depth < len(d.chose) \{$ 
     $d.chose[depth][d.name[best]] = \text{true}$ 
   $\}$ 
  return  $best$ 
 $\}$ 

```

18. A solution is a set of options, one per item of the left half; read off the right half and it is a permutation.

$\langle \text{Record a solution 18} \rangle \equiv$

```

 $d.count ++$ 
if  $d.keep \{$ 
   $n := d.items/2$ 
   $p := make([], \text{int}, n)$ 
  for  $k := 0; k < n; k ++ \{$ 
     $o := d.opts[d.path[k]]$ 
     $p[o[0] - 1] = o[1] - n$ 
   $\}$ 
   $d.sols = append(d.sols, p)$ 
 $\}$ 

```

This code is used in section 14.

19. Bounded permutation problems. The problem for $a_1 \dots a_n$ has $2n$ items $X_1, \dots, X_n, Y_1, \dots, Y_n$ and the $a_1 + \dots + a_n$ options ' $X_j Y_k$ ' for $1 \leq k \leq a_j$. The order of the items is the order in which they are declared, and it decides the ties in step X3; I put the X 's first, as the exercise does.

(Functions 8) +≡

```

func problem(a []int) *DLX {
    n := len(a)
    names := make([]string, 0, 2 * n)
    for j := 1; j ≤ n; j++ {
        names = append(names, fmt.Sprintf("X%d", j))
    }
    for k := 1; k ≤ n; k++ {
        names = append(names, fmt.Sprintf("Y%d", k))
    }
    var opts [][]int
    for j := 1; j ≤ n; j++ {
        for k := 1; k ≤ a[j - 1]; k++ {
            opts = append(opts, []int{j, n + k})
        }
    }
    d := newDLX(names, opts)
    d.path = make([]int, n + 1)
    return d
}

```

20. Three assumptions come free with the problem, as page 103 explains: a is nondecreasing, $a_j \geq j$, and $a_n \leq n$. A sequence that breaks the middle one has no solutions at all, so the sweeps stay inside the canonical form.

(Functions 8) +≡

```

func canonical(a []int) bool {
    n := len(a)
    for j := 1; j ≤ n; j++ {
        if a[j - 1] < j ∨ a[j - 1] > n {
            return false
        }
        if j > 1 ∧ a[j - 2] > a[j - 1] {
            return false
        }
    }
    return true
}

```

21. $\langle \text{Functions 8} \rangle + \equiv$

```

func digitSeq(s string) []int {
    a := make([]int, len(s))
    for i, c := range s {
        if c < '1' ∨ c > '9' {
            log.Fatalf("bad sequence %q", s)
        }
        a[i] = int(c - '0')
    }
    if ¬canonical(a) {
        log.Fatalf("%q is not a canonical bounded permutation problem", s)
    }
    return a
}

```

22. The dual counts, for each k , how many of the a_j reach as high as $n + 1 - k$. Rotating the Young diagram of a by 180° turns rows into columns and back to rows, which is why $b_n \dots b_1$ is the conjugate partition of $a_n \dots a_1$ and why dualizing twice gives a again.

$\langle \text{Functions 8} \rangle + \equiv$

```

func dual(a []int) []int {
    n := len(a)
    b := make([]int, n)
    for k := 1; k ≤ n; k++ {
        for _, aj := range a {
            if aj ≥ n + 1 - k {
                b[k - 1]++
            }
        }
    }
    return b
}

```

23. $\langle \text{Functions 8} \rangle + \equiv$

```

func show(a []int) string {
    var b strings.Builder
    for _, x := range a {
        fmt.Fprintf(&b, "%d", x)
    }
    return b.String()
}

```

24. Part (a) wants one dual, and gets three checks: the printed answer, the involution, and the canonical form.

$\langle \text{Take the dual, part (a) 24} \rangle \equiv$

```

b := dual(a)
fmt.Printf("(a) a=%s, dual b=%s (answer: 135778899, agrees: %v)\n",
    show(a), show(b), show(b) == "135778899")
fmt.Printf("b is canonical: %v; dual of the dual is a: %v\n",
    canonical(b), show(dual(b)) == show(a))

```

This code is used in section 4.

25. Inverses. Part (b) says two things. The options correspond: with $\bar{k} = n + 1 - k$, ‘ $X_j Y_k$ ’ is a dual option if and only if ‘ $Y_j X_k$ ’ is an original one. And so do the solutions: $q_1 \dots q_n$ is the inverse of an original solution if and only if $\bar{q}_n \dots \bar{q}_1$ is a dual solution.

The first half is a statement about the sequences alone, and it comes out of the definition. Since a is nondecreasing, b_j counts a final segment of the a ’s: $b_j = n - t + 1$ where t is least with $a_t \geq n + 1 - j$. Then $k \leq b_j$ says $n + 1 - k \geq t$, which says $a_{n+1-k} \geq n + 1 - j$. That is the claim.

⟨Functions 8⟩ +≡

```

func optionsMatch(a []int) bool {
  n := len(a)
  b := dual(a)
  for j := 1; j ≤ n; j++ {
    for k := 1; k ≤ n; k++ {
      if (k ≤ b[j − 1]) ≠ (n + 1 − j ≤ a[n − k]) {
        return false
      }
    }
  }
  return true
}

```

26. The second half I check by enumeration, which is why this sweep stops sooner than the others. Take every solution of the original problem, invert the permutation, reverse it and complement each entry; the result should be exactly the set of solutions of the dual, with nothing left over on either side.

⟨Functions 8⟩ +≡

```

func inversesMatch(a []int) bool {
  n := len(a)
  ⟨Collect the solutions of a and of its dual 27⟩
  for -, p := range orig {
    inv := make([]int, n)
    for j := 1; j ≤ n; j++ {
      inv[p[j − 1] − 1] = j
    }
    r := make([]int, n)
    for j := 1; j ≤ n; j++ {
      r[j − 1] = n + 1 − inv[n − j]
    }
    if ¬want[show(r)] {
      return false
    }
    delete(want, show(r))
  }
  return len(want) ≡ 0
}

```

27. $\langle$ Collect the solutions of a and of its dual 27 $\rangle \equiv$

```

d := problem(a)
d.keep = true
d.search(0)
orig := d.sols
e := problem(dual(a))
e.keep = true
e.search(0)
want := map[string]bool{ }
for  $\_, q$  := range e.sols {
    want[show(q)] = true
}

```

This code is used in section 26.

28. $\langle$ Match the solutions with their inverses, part (b) 28 $\rangle \equiv$

```

fmt.Printf("(b)  $\_a = \_s$ :  $\_options$   $\_correspond$ :  $\_v$ ;  $\_solutions$   $\_invert$ :  $\_v$ \n",
    show(a), optionsMatch(a), inversesMatch(a))
bad, seqs, far := 0, 0, min(*upto, 7)
for  $n := 1$ ;  $n \leq far$ ;  $n++$  {
    for  $\_, c$  := range allSeqs(n) {
        seqs++
        if  $\neg optionsMatch(c) \vee \neg inversesMatch(c)$  {
            bad++
            fmt.Println(" $\_$ FAILS:", show(c))
        }
    }
}
fmt.Printf(" $\_$ every  $\_problem$   $\_with$   $\_n \leq \_d$ :  $\_d$   $\_sequences$ ,  $\_d$   $\_failures$ \n",
    far, seqs, bad)

```

This code is used in section 4.

29. Preparing the subproblems. Answer (c) says the root costs $1 + a_1(n + 1)$ updates, “because each Y_k for $1 \leq k \leq a_1$ appears in n options.” Follow the algorithm: covering X_1 unlinks X_1 itself, one update, and hides the single Y node of each of its a_1 options, a_1 more. Then each of the a_1 branches covers its Y_k , which now has $n - 1$ options left, for $1 + (n - 1) = n$ updates apiece. Altogether $1 + a_1 + a_1n$.

The program measures it instead of arguing: *prep* accumulates exactly the updates made at depth 0, and nothing from inside the subtrees.

⟨ Count the updates at the root, part (c) 29 ⟩ $\equiv$

```

fmt.Println("(c) updates spent preparing the depth-1 subproblems")
for _, c := range []([]int) { a, digitSeq("123456789"), digitSeq("999999999"),
  digitSeq("234456789"), digitSeq("22") } {
  d := problem(c)
  d.leftmost = true // the branch on  $X_1$  that part (c) assumes
  d.search(0)
  n := len(c)
  fmt.Printf("a=%-9s measured %5d 1+a_1(n+1)=%5d %v\n",
    show(c), d.prep, 1 + c[0] * (n + 1), d.prep  $\equiv$  uint64(1 + c[0] * (n + 1)))
}

```

This code is used in section 4.

30. How many solutions. Answer (d) is $a_1(a_2 - 1)(a_3 - 2) \dots (a_n - n + 1)$: having chosen $p_1, \dots, p_{j-1}$, all of them at most $a_{j-1} \leq a_j$, the values still open to p_j number $a_j - j + 1$ whatever the earlier choices were. The same count applied to the dual must give the same answer, which the exercise calls not an obvious fact, and which part (b) explains.

The partial products are the ones that count the search tree, so they get a name of their own.

⟨ Functions 8 ⟩ +≡

```
func bigPi(a []int, j int) *big.Int {
    p := big.NewInt(1)
    for i := 1; i ≤ j; i++ {
        p.Mul(p, big.NewInt(int64(a[i-1] - i + 1)))
    }
    return p
}
```

31. For a second opinion on the number of solutions I hand the same problem to `ssxcc`, the engine of this repository, in the DLX text format. How many solutions a problem has does not depend on the algorithm that finds them.

⟨ Functions 8 ⟩ +≡

```
func dlxText(a []int) string {
    n := len(a)
    var b strings.Builder
    for j := 1; j ≤ n; j++ {
        fmt.Fprintf(&b, "X%d", j)
    }
    for k := 1; k ≤ n; k++ {
        fmt.Fprintf(&b, "Y%d", k)
    }
    b.WriteByte('\n')
    for j := 1; j ≤ n; j++ {
        for k := 1; k ≤ a[j-1]; k++ {
            fmt.Fprintf(&b, "X%dY%d\n", j, k)
        }
    }
    return b.String()
}
```

32. ⟨ Functions 8 ⟩ +≡

```
func ssxccCount(a []int) uint64 {
    x := cells.NewXCC()
    res := x.Dance(strings.NewReader(dlxText(a)))
    var n uint64
    for range res.Solutions {
        n++
    }
    return n
}
```

33. $\langle$ Count the solutions, part (d) 33 $\rangle \equiv$

```

fmt.Println("(d) solutions, by the formula and by three searches")
for _, c := range []([]int) { a, digitSeq("123456789"), digitSeq("999999999"),
  digitSeq("2466889999") } {
  n := len(c)
  d := problem(c)
  d.search(0)
  pa, pb := bigPi(c, n), bigPi(dual(c), n)
  fmt.Printf("a=%-9s formula%-7sdual%-7sAlgorithmX%-7d" +
    "ssxcc%-7d\n", show(c), pa, pb, d.count, ssxccCount(c),
    pa.Cmp(pb) == 0 ^ pa.String() == fmt.Sprint(d.count) ^
    d.count == ssxccCount(c))
}

```

This code is used in section 4.

34. The total. Answer (e) reads $1 + (\sum_{j=1}^n (n+3-j)\Pi_j) - \Pi_n$, where $\Pi_j = \prod_{i=1}^j (a_i - i + 1)$. It follows from (c). Each of the Π_{j-1} nodes at depth $j-1$ faces a bounded permutation problem on $n-j+1$ items whose first bound is $a_j - j + 1$, so by (c) it spends $1 + (a_j - j + 1)(n-j+2)$ updates getting its children ready. Summing,

$$\sum_{j=1}^n \Pi_{j-1} + \sum_{j=1}^n (n-j+2)\Pi_j = 1 + \sum_{j=1}^n \Pi_j - \Pi_n + \sum_{j=1}^n (n-j+2)\Pi_j,$$

which is the printed formula.

⟨ Functions 8 ⟩ +≡

```

func total(a []int) *big.Int {
    n := len(a)
    t := big.NewInt(1)
    for j := 1; j ≤ n; j++ {
        t.Add(t, new(big.Int).Mul(big.NewInt(int64(n+3-j)), bigPi(a, j)))
    }
    return t.Sub(t, bigPi(a, n))
}

```

35. ⟨ Count all of the updates, part (e) 35 ⟩ ≡

```

fmt.Println("(e) all the updates, under the assumption of (e)")
for _, c := range [](int) { a, digitSeq("123456789"), digitSeq("999999999"),
    digitSeq("246688999"), digitSeq("234567899") }{
    d := problem(c)
    d.leftmost = true
    d.search(0)
    t := total(c)
    fmt.Printf("a=%-9s measured %-10d formula %-10s %v\n",
        show(c), d.updates, t, t.String()) ≡ fmt.Sprint(d.updates)
}

```

This code is used in section 4.

36. Three families. Parts (f), (g) and (h) evaluate the formula on three sequences that grow with n : every permutation, $a_j = \min(j+1, n)$, and $a_j = \min(2j, n)$. All three satisfy the assumption of part (e)—the sweep in the last section shows that the heuristic really does branch on X_j throughout—so the formula and the search must agree, and the interest is in what the closed forms say.

⟨ Functions 8 ⟩ +≡

```
func family(kind byte, n int) []int {
    a := make([]int, n)
    for j := 1; j ≤ n; j++ {
        switch kind {
        case 'f':
            a[j-1] = n
        case 'g':
            a[j-1] = min(j+1, n)
        case 'h':
            a[j-1] = min(2*j, n)
        }
    }
    return a
}
```

37. Answer (f) is $1 + (\sum_{j=1}^n (n+3-j)n^j) - n!$, which is about $(4e-1)n!$; here Π_j is the falling factorial power. Answer (g) is $6 \cdot 2^n - 2n - 7$, since $\Pi_j = 2^j$ for $j < n$ and $\Pi_n = 2^{n-1}$. Answer (h) gives Π_n and says the updates per solution again approach $4e-1$.

⟨ Functions 8 ⟩ +≡

```
func closedForm(kind byte, n int) *big.Int {
    switch kind {
    case 'g': // 6 · 2^n - 2n - 7
        t := new(big.Int).Lsh(big.NewInt(6), uint(n))
        return t.Sub(t, big.NewInt(int64(2*n+7)))
    case 'h': // the Π_n of answer (h), with its factorials
        p := new(big.Int).MulRange(1, int64((n+1)/2))
        return p.Mul(p, new(big.Int).MulRange(1, int64((n+2)/2)))
    }
    return new(big.Int).MulRange(1, int64(n)) // n!, the solutions in (f)
}
```

38. ⟨ Functions 8 ⟩ +≡

```
func ratio(x, y *big.Int) float64 {
    f, _ := new(big.Rat).SetFrac(x, y).Float64()
    return f
}
```

39. Each family gets a table: the updates, the solutions, their quotient, and wherever the search is still small enough to run, the count Algorithm X actually makes.

⟨ Evaluate the three families, parts (f), (g), (h) 39 ⟩ ≡

```

for _, kind := range []byte { 'f', 'g', 'h' } {
    ⟨ Announce the family 40 ⟩
    for n := 1; n ≤ *top; n++ {
        c := family(kind, n)
        t, sols := total(c), bigPi(c, n)
        line := fmt.Sprintf("_n=%2d_updates%-14s_solutions%-11s" +
            "_per_solution%-9.6f", n, t, sols, ratio(t, sols))
        ⟨ Compare with the closed form and with a real search 41 ⟩
        fmt.Println(line)
    }
}
⟨ Look at the series of answer (h) 42 ⟩

```

This code is used in section 4.

40. ⟨ Announce the family 40 ⟩ ≡

```

switch kind {
case 'f':
    fmt.Println("(f)_a_j=_n,_all_permutations")
case 'g':
    fmt.Println("(g)_a_j=_min(j+1,n)")
case 'h':
    fmt.Println("(h)_a_j=_min(2j,n)")
}

```

This code is used in section 39.

41. In family (g) the closed form is the update total; in (f) and (h) it is the number of solutions. Running the search itself costs Π_n time, so I stop when that gets silly.

⟨ Compare with the closed form and with a real search 41 ⟩ ≡

```

cf := closedForm(kind, n)
if kind ≡ 'g' {
    line += fmt.Sprintf("_6*2^n-2n-7_v", cf.Cmp(t) ≡ 0)
} else {
    line += fmt.Sprintf("_closed_form_v", cf.Cmp(sols) ≡ 0)
}
if sols.IsInt64() ∧ sols.Int64() < 100000 {
    d := problem(c)
    d.search(0)
    line += fmt.Sprintf("_search_v",
        t.String() ≡ fmt.Sprint(d.updates) ∧ d.count ≡ uint64(sols.Int64()))
}

```

This code is used in section 39.

42. The one place where the printed answer and the arithmetic part company is the series in (h). Divide the total by Π_n and let $i = n - j$: the terms with $j > n/2$ contribute $\sum_i (i + 3)/i!$, the terms with $j \leq n/2$ are the $O(n^2/(n/2)!)$ remainder, and the $-\Pi_n$ contributes -1 . So the series is

$$(3 - 1) + 4/1! + 5/2! + \dots = 4e - 1,$$

with 2 in front, not 6. The printed 6 would make the sum $4e + 3$, which is not what the same line says it converges to.

(Look at the series of answer (h) [42](#)) $\equiv$

```

sum, fact := 0.0, 1.0
for i := 1; i ≤ 20; i++ {
    fact *= float64(i)
    sum += float64(i + 3)/fact
}
fmt.Printf("(h) 4e-1=%.9f; 2+4/1!+5/2!+...=%.9f; 6+4/1!+5/2!+...=%.9f\n",
    4 * math.E - 1, 2 + sum, 6 + sum)
c := family('h', 200)
fmt.Printf("the ratio at n=200 is %.9f\n", ratio(total(c), bigPi(c, 200)))

```

This code is used in section [39](#).

43. What the heuristic really does. Part (i) grants that the assumption of (e) can fail, and says why: if $b_1 < a_1$ then Y_n has fewer options than X_1 , so the first branch is on Y_n , not X_1 , and the root costs $1 + b_1(n + 1)$ updates. It closes with an example: “The example problem in (a) branches on Y_9 , then X_2 , then Y_8 , etc.”

Two of those three claims survive the reading. The count $1 + b_1(n + 1)$ is right every time. The branching sequence is not.

⟨ Watch the heuristic choose, part (i) 43 ⟩ $\equiv$
 ⟨ Show the items the heuristic picks 44 ⟩
 ⟨ Check the root cost when the dual wins 46 ⟩
 ⟨ Check the general recursion 49 ⟩

This code is used in section 4.

44. Watching is a matter of recording, at each depth, the names of the items that step X3 chose there—a set, because different branches may reach different subproblems.

⟨ Show the items the heuristic picks 44 ⟩ $\equiv$

```

n := len(a)
d := problem(a)
d.chose = make([]map[string]bool, n)
for i := range d.chose {
    d.chose[i] = map[string]bool{ }
}
d.search(0)
fmt.Printf("(i) a=%s: the items step X3 branches on\n", show(a))
for i, s := range d.chose {
    var names []string
    for k := range s {
        names = append(names, k)
    }
    sort.Strings(names)
    fmt.Printf("    depth%d: %s\n", i, strings.Join(names, " "))
}
fmt.Printf("    Algorithm X makes %d updates and %d nodes here;" +
    " the formula of (e) says %s\n", d.updates, d.nodes, total(a))

```

This code is used in section 43.

45. The rule behind the sequence is easy to state. Because a is nondecreasing, the shortest X item is X_1 with a_1 options; because b is nondecreasing, the shortest Y item has b_1 options. Ties go to the X , which is declared first. So the branch is on an X exactly when $a_1 \leq b_1$.

For 246677889 that gives Y_9 , then X_1 , then Y_8 , then X_2 : the two sides alternate while the bounds stay tight. Answer (i) omits the X_1 and puts X_2 where Y_8 belongs.

46. There is a second, smaller slip in the same sentence. The shortest Y item is Y_n only if no other Y ties with it, and ties do happen: if the a_j skip a value, several Y 's have the same length and step X3 takes the leftmost. The sweep below counts how often. The update total is unaffected, because the tied items are the ones whose options all lead to X 's of full length n .

⟨ Check the root cost when the dual wins 46 ⟩ $\equiv$

```

    tried, notLast, wrong := 0, 0, 0
    first := ""
    for n := 1; n ≤ *upto + 1; n++ {
        for _, c := range allSeqs(n) {
            b1 := dual(c)[0]
            if b1 ≥ c[0] {
                continue
            }
            tried++
        }
        ⟨ Run the root and check what it chose 47 ⟩
    }
}
fmt.Printf("problems with b_1 < a_1 (n ≤ %d): %d of them, %d cost" +
    " something other than 1+b_1(n+1)\n", *upto + 1, tried, wrong)
fmt.Printf("%d branch on a_Y other than Y_n; the first is a = %s\n",
    notLast, first)

```

This code is used in section 43.

47. ⟨ Run the root and check what it chose 47 ⟩ $\equiv$

```

    d := problem(c)
    d.chose = []map[string]bool{ {} }
    d.search(0)
    if d.prep ≠ uint64(1 + b1 * (n + 1)) {
        wrong++
    }
    if ¬d.chose[0][fmt.Sprintf("Y%d", n)] {
        notLast++
        if first ≡ "" {
            first = show(c)
        }
    }
}

```

This code is used in section 46.

48. “How can the total updates be calculated correctly in general?” By the same recursion, with the choice put in. Let b_1 be the number of a_i equal to n . If $a_1 \leq b_1$ the algorithm branches on X_1 , spends $1 + a_1(n + 1)$ updates, and hands each of a_1 children the problem $(a_2 - 1) \dots (a_n - 1)$. Otherwise it branches on the shortest Y , spends $1 + b_1(n + 1)$, and hands each of b_1 children the problem $\min(a_1, n - 1) \dots \min(a_{n-1}, n - 1)$. Both children are the same for every branch, which is what makes a recursion on the sequence alone possible; and both are canonical.

⟨ Functions 8 ⟩ +≡

```

func mrvTotal(a []int) *big.Int {
    n := len(a)
    if n == 0 {
        return big.NewInt(0)
    }
    b1, sub := 0, make([]int, n - 1)
    for _, x := range a {
        if x == n {
            b1++
        }
    }
    branches := a[0]
    if a[0] ≤ b1 {
        for i := 1; i < n; i++ {
            sub[i - 1] = a[i] - 1
        }
    } else {
        branches = b1
        for i := 0; i < n - 1; i++ {
            sub[i] = min(a[i], n - 1)
        }
    }
    t := big.NewInt(int64(1 + branches * (n + 1)))
    return t.Add(t, new(big.Int).Mul(big.NewInt(int64(branches)), mrvTotal(sub)))
}

```

49. ⟨ Check the general recursion 49 ⟩ ≡

```

fmt.Printf("the recursion gives %s for a = %s\n", mrvTotal(a), show(a))
seqs, bad := 0, 0
for n := 1; n ≤ *upto; n++ {
    for _, c := range allSeqs(n) {
        d := problem(c)
        d.search(0)
        seqs++
        if mrvTotal(c).String() ≠ fmt.Sprintf(d.updates) {
            bad++
            fmt.Println("FAILS:", show(c))
        }
    }
}
fmt.Printf("every problem with n ≤ %d: %d sequences, %d failures\n",
    *upto, seqs, bad)

```

This code is used in section 43.

50. Sweeps. Every canonical sequence up to a given length, in lexicographic order.

⟨ Functions 8 ⟩ +≡

```

func allSeqs(n int) [][]int {
    var out [][]int
    cur := make([]int, n)
    var rec func(j, prev int)
    rec = func(j, prev int) {
        if j > n {
            out = append(out, append([]int( $\Lambda$ ), cur...))
            return
        }
        for v := max(j, prev); v ≤ n; v++ {
            cur[j - 1] = v
            rec(j + 1, v)
        }
    }
    rec(1, 1)
    return out
}

```

51. How many such sequences are there? A nondecreasing $a_1 \dots a_n$ with $j \leq a_j \leq n$ is a ballot sequence read sideways, so there are C_n of them, the Catalan number; the sweep confirms 1, 2, 5, 14, 42, 132, 429 as it goes.

⟨ Functions 8 ⟩ +≡

```

func catalan(n int) *big.Int {
    c := new(big.Int).Binomial(int64(2 * n), int64(n))
    return c.Div(c, big.NewInt(int64(n + 1)))
}

```

52. The last mode puts the four formulas side by side on every small problem at once: the solution count of (d), the update total of (e) under its assumption, the recursion of (i) against the real search, and the question of how often the assumption of (e) holds at all.

⟨ Sweep every small problem 52 ⟩ ≡

```

fmt.Printf("sweep: every canonical a_1...a_n with n ≤ %d\n", *upto)
for n := 1; n ≤ *upto; n++ {
    seqs, badD, badE, badI, differ := 0, 0, 0, 0, 0
    for _, c := range allSeqs(n) {
        seqs++
        ⟨ Compare the formulas on c 53 ⟩
    }
    fmt.Printf("n=%d: %5d sequences (Catalan: %v), %d/%d/%d failures" +
        " in (d)/(e)/(i), %d where the heuristic leaves the assumption" +
        " of (e)\n", n, seqs, catalan(n).String() ≡ fmt.Sprint(seqs),
        badD, badE, badI, differ)
}

```

This code is used in section 4.

53. Running the same problem twice, once with the heuristic and once with the leftmost rule, is what separates the assumption of part (e) from what Algorithm X does. The two agree on the number of solutions, never on more.

⟨ Compare the formulas on *c* 53 ⟩ ≡

```

x := problem(c)
x.leftmost = true
x.search(0)
m := problem(c)
m.search(0)
if bigPi(c, n).String() ≠ fmt.Sprint(m.count) {
    badD ++
}
if total(c).String() ≠ fmt.Sprint(x.updates) {
    badE ++
}
if mrvtotal(c).String() ≠ fmt.Sprint(m.updates) {
    badI ++
}
if x.updates ≠ m.updates {
    differ ++
}

```

This code is used in section 52.

54. Index.

- [a](#): [5](#), [19](#), [20](#), [21](#), [22](#), [23](#), [24](#), [25](#), [26](#), [27](#), [28](#), [29](#), [30](#),
[31](#), [32](#), [33](#), [34](#), [35](#), [36](#), [44](#), [48](#), [49](#).
Add: [34](#), [48](#).
aj: [22](#).
allSeqs: [28](#), [46](#), [49](#), [50](#), [52](#).
append: [8](#), [9](#), [18](#), [19](#), [44](#), [50](#).
b: [22](#), [23](#), [24](#), [25](#), [31](#).
b1: [46](#), [47](#), [48](#).
bad: [28](#), [49](#).
badD: [52](#), [53](#).
badE: [52](#), [53](#).
badI: [52](#), [53](#).
best: [17](#).
big: [30](#), [34](#), [37](#), [38](#), [48](#), [51](#).
bigPi: [30](#), [33](#), [34](#), [39](#), [42](#), [53](#).
Binomial: [51](#).
bool: [7](#), [20](#), [25](#), [26](#), [27](#), [44](#), [47](#).
branches: [48](#).
branchItem: [14](#), [17](#).
Builder: [23](#), [31](#).
byte: [36](#), [37](#), [39](#).
c: [21](#), [28](#), [29](#), [33](#), [35](#), [39](#), [41](#), [42](#), [46](#), [47](#), [49](#), [51](#), [52](#),
[53](#).
canonical: [20](#), [21](#), [24](#).
catalan: [51](#), [52](#).
cells: [4](#), [32](#).
cf: [41](#).
chose: [7](#), [17](#), [44](#), [47](#).
closedForm: [37](#), [41](#).
Cmp: [33](#), [41](#).
count: [7](#), [18](#), [33](#), [41](#), [53](#).
cover: [6](#), [7](#), [10](#), [14](#), [15](#).
cur: [50](#).
d: [8](#), [9](#), [10](#), [11](#), [12](#), [13](#), [14](#), [15](#), [16](#), [17](#), [18](#), [19](#), [27](#), [29](#),
[33](#), [35](#), [41](#), [44](#), [47](#), [49](#).
Dance: [32](#).
delete: [26](#).
depth: [14](#), [15](#), [17](#).
diff: [52](#), [53](#).
digits: [5](#).
digitSeq: [5](#), [21](#), [29](#), [33](#), [35](#).
Div: [51](#).
dlink: [7](#), [9](#), [10](#), [11](#), [13](#), [14](#), [16](#).
dlx: [7](#), [8](#), [10](#), [12](#), [14](#), [17](#), [19](#).
dlxText: [31](#), [32](#).
dual: [22](#), [24](#), [25](#), [27](#), [33](#), [46](#).
E: [42](#).
e: [27](#).
f: [38](#).
fact: [42](#).
family: [36](#), [39](#), [42](#).
far: [28](#).
Fatalf: [4](#), [21](#).
first: [9](#), [46](#), [47](#).
flag: [5](#).
Float64: [38](#).
float64: [38](#), [42](#).
fmt: [19](#), [23](#), [24](#), [28](#), [29](#), [31](#), [33](#), [35](#), [39](#), [40](#), [41](#), [42](#),
[44](#), [46](#), [47](#), [49](#), [52](#), [53](#).
Fprintf: [23](#), [31](#).
hide: [6](#), [7](#), [9](#).
i: [8](#), [9](#), [10](#), [12](#), [14](#), [15](#), [17](#), [21](#), [30](#), [42](#), [44](#), [48](#).
int: [7](#), [8](#), [9](#), [10](#), [12](#), [14](#), [17](#), [18](#), [19](#), [20](#), [21](#), [22](#), [23](#), [25](#),
[26](#), [29](#), [30](#), [31](#), [32](#), [33](#), [34](#), [35](#), [36](#), [37](#), [48](#), [50](#), [51](#).
Int: [5](#), [30](#), [34](#), [37](#), [38](#), [48](#), [51](#).
Int64: [41](#).
int64: [30](#), [34](#), [37](#), [48](#), [51](#).
inv: [26](#).
inversesMatch: [26](#), [28](#).
IsInt64: [41](#).
items: [7](#), [8](#), [18](#).
j: [15](#), [16](#), [19](#), [20](#), [25](#), [26](#), [30](#), [31](#), [34](#), [36](#), [50](#).
Join: [44](#).
k: [9](#), [18](#), [19](#), [22](#), [25](#), [31](#), [44](#).
keep: [7](#), [18](#), [27](#).
kind: [36](#), [37](#), [39](#), [40](#), [41](#).
 Knuth, Donald Ervin: [2](#).
l: [10](#), [12](#).
least: [17](#).
leftmost: [7](#), [17](#), [29](#), [35](#), [53](#).
len: [8](#), [9](#), [17](#), [19](#), [20](#), [21](#), [22](#), [25](#), [26](#), [29](#), [31](#), [33](#), [34](#),
[44](#), [48](#).
line: [39](#), [41](#).
llink: [7](#), [8](#), [10](#), [12](#).
log: [4](#), [21](#).
Lsh: [37](#).
m: [4](#), [53](#).
main: [4](#).
make: [8](#), [9](#), [18](#), [19](#), [21](#), [22](#), [26](#), [36](#), [44](#), [48](#), [50](#).
mark: [14](#), [15](#).
math: [42](#).
max: [50](#).
min: [28](#), [36](#), [48](#).
mode: [5](#).

- modes*: 4, [5](#).
mrvTotal: [48](#), [49](#), [53](#).
Mul: [30](#), [34](#), [37](#), [48](#).
MulRange: [37](#).
n: [8](#), [9](#), [18](#), [19](#), [20](#), [22](#), [25](#), [26](#), [28](#), [29](#), [31](#), [32](#), [33](#), [34](#),
[36](#), [37](#), [39](#), [41](#), [44](#), [46](#), [47](#), [48](#), [49](#), [50](#), [51](#), [52](#), [53](#).
name: [7](#), [8](#), [17](#).
names: [8](#), [19](#), [44](#).
new: [34](#), [37](#), [38](#), [48](#), [51](#).
newDLX: [8](#), [19](#).
NewInt: [30](#), [34](#), [37](#), [48](#), [51](#).
NewReader: [32](#).
NewXCC: [32](#).
nodes: [7](#), [14](#), [44](#).
notLast: [46](#), [47](#).
o: [9](#), [18](#).
optionsMatch: [25](#), [28](#).
optOf: [7](#), [9](#), [14](#).
opts: [7](#), [8](#), [9](#), [18](#), [19](#).
orig: [26](#), [27](#).
out: [50](#).
p: [10](#), [11](#), [12](#), [13](#), [15](#), [16](#), [18](#), [26](#), [30](#), [37](#).
pa: [33](#).
Parse: [5](#).
path: [7](#), [14](#), [18](#), [19](#).
pb: [33](#).
prep: [7](#), [14](#), [15](#), [29](#), [47](#).
prev: [50](#).
Printf: [24](#), [28](#), [29](#), [33](#), [35](#), [42](#), [44](#), [46](#), [49](#), [52](#).
Println: [28](#), [29](#), [33](#), [35](#), [39](#), [40](#), [49](#).
problem: [19](#), [27](#), [29](#), [33](#), [35](#), [41](#), [44](#), [47](#), [49](#), [53](#).
q: [11](#), [13](#), [27](#).
r: [10](#), [12](#), [26](#).
Rat: [38](#).
ratio: [38](#), [39](#), [42](#).
rec: [50](#).
res: [32](#).
rlink: [7](#), [8](#), [10](#), [12](#), [14](#), [17](#).
s: [21](#), [44](#).
search: [14](#), [27](#), [29](#), [33](#), [35](#), [41](#), [44](#), [47](#), [49](#), [53](#).
seqs: [28](#), [49](#), [52](#).
SetFrac: [38](#).
show: [23](#), [24](#), [26](#), [27](#), [28](#), [29](#), [33](#), [35](#), [44](#), [47](#), [49](#).
size: [7](#), [9](#), [11](#), [13](#), [17](#).
sols: [7](#), [18](#), [27](#), [39](#), [41](#).
Solutions: [32](#).
sort: [44](#).
Sprint: [33](#), [35](#), [41](#), [49](#), [52](#), [53](#).
Sprintf: [19](#), [39](#), [41](#), [47](#).
sscccCount: [32](#), [33](#).
String: [5](#), [23](#), [31](#), [33](#), [35](#), [41](#), [49](#), [52](#), [53](#).
string: [5](#), [7](#), [8](#), [19](#), [21](#), [23](#), [27](#), [31](#), [44](#), [47](#).
Strings: [44](#).
strings: [23](#), [31](#), [32](#), [44](#).
Sub: [34](#), [37](#).
sub: [48](#).
sum: [42](#).
t: [34](#), [35](#), [37](#), [39](#), [41](#), [48](#).
top: [5](#), [7](#), [9](#), [11](#), [13](#), [15](#), [16](#), [39](#).
total: [34](#), [35](#), [39](#), [42](#), [44](#), [53](#).
tried: [46](#).
u: [9](#), [11](#), [13](#).
uint: [37](#).
uint64: [7](#), [29](#), [32](#), [41](#), [47](#).
ulink: [7](#), [9](#), [11](#), [12](#), [13](#), [15](#).
uncover: [7](#), [12](#), [14](#), [16](#).
unhide: [7](#).
updates: [7](#), [10](#), [11](#), [14](#), [15](#), [35](#), [41](#), [44](#), [49](#), [53](#).
upto: [5](#), [28](#), [46](#), [49](#), [52](#).
v: [50](#).
w: [11](#), [13](#).
want: [26](#), [27](#).
WriteByte: [31](#).
wrong: [46](#), [47](#).
x: [9](#), [11](#), [13](#), [14](#), [15](#), [16](#), [23](#), [32](#), [38](#), [48](#), [53](#).
y: [38](#).

- ⟨ Announce the family [40](#) ⟩ Used in section [39](#)
- ⟨ Check the general recursion [49](#) ⟩ Used in section [43](#)
- ⟨ Check the root cost when the dual wins [46](#) ⟩ Used in section [43](#)
- ⟨ Collect the solutions of a and of its dual [27](#) ⟩ Used in section [26](#)
- ⟨ Compare the formulas on c [53](#) ⟩ Used in section [52](#)
- ⟨ Compare with the closed form and with a real search [41](#) ⟩ Used in section [39](#)
- ⟨ Count all of the updates, part (e) [35](#) ⟩ Used in section [4](#)
- ⟨ Count the solutions, part (d) [33](#) ⟩ Used in section [4](#)
- ⟨ Count the updates at the root, part (c) [29](#) ⟩ Used in section [4](#)
- ⟨ Cover the other items of option x [15](#) ⟩ Used in section [14](#)
- ⟨ Declarations [7](#) ⟩ Used in section [4](#)
- ⟨ Evaluate the three families, parts (f), (g), (h) [39](#) ⟩ Used in section [4](#)
- ⟨ Functions [8](#) ⟩ Used in section [4](#)
- ⟨ Hide the other items of the option through p [11](#) ⟩ Used in section [10](#)
- ⟨ Lay out the node array [9](#) ⟩ Used in section [8](#)
- ⟨ Look at the series of answer (h) [42](#) ⟩ Used in section [39](#)
- ⟨ Match the solutions with their inverses, part (b) [28](#) ⟩ Used in section [4](#)
- ⟨ Read the command line [5](#) ⟩ Used in section [4](#)
- ⟨ Record a solution [18](#) ⟩ Used in section [14](#)
- ⟨ Run the root and check what it chose [47](#) ⟩ Used in section [46](#)
- ⟨ Show the items the heuristic picks [44](#) ⟩ Used in section [43](#)
- ⟨ Sweep every small problem [52](#) ⟩ Used in section [4](#)
- ⟨ Take the dual, part (a) [24](#) ⟩ Used in section [4](#)
- ⟨ Uncover the other items of option x [16](#) ⟩ Used in section [14](#)
- ⟨ Unhide the other items of the option through p [13](#) ⟩ Used in section [12](#)
- ⟨ Watch the heuristic choose, part (i) [43](#) ⟩ Used in section [4](#)

Bounded permutation problems

	Section	Page
Introduction	1	1
Algorithm X, with its updates counted	6	5
Bounded permutation problems	19	10
Inverses	25	12
Preparing the subproblems	29	14
How many solutions	30	15
The total	34	17
Three families	36	18
What the heuristic really does	43	21
Sweeps	50	24
Index	54	26